

BUILD-SUPABASE.md

Partie C.10 — Triggers PostgreSQL

Version : V1

Statut : Référence officielle des triggers PostgreSQL

1. Objectif

Cette section définit les règles officielles de conception et d'utilisation des triggers PostgreSQL dans Vitala.

Les triggers permettent :

- d'automatiser des traitements techniques ;
- de garantir la cohérence des données ;
- de simplifier certaines opérations répétitives.

Ils ne remplacent jamais les Domaines métier ni les APIs.

2. Principes directeurs

Tous les triggers doivent respecter les principes suivants :

- Technical First
- Domain First
- Single Responsibility
- Transparence
- Performance
- Documentation obligatoire

Chaque trigger doit avoir une responsabilité unique et clairement identifiée.

3. Cas d'utilisation autorisés

Les triggers peuvent être utilisés pour :

- mettre à jour automatiquement `updated_at` ;
- compléter des métadonnées techniques ;
- enregistrer des informations d'audit ;
- maintenir des tables techniques ;

- déclencher des notifications techniques ;
- alimenter des journaux d'événements.

Ils doivent rester simples et déterministes.

4. Cas d'utilisation interdits

Les triggers ne doivent jamais :

- implémenter des règles métier ;
- prendre des décisions fonctionnelles ;
- modifier plusieurs domaines métier ;
- appeler des services externes ;
- exécuter des workflows complexes ;
- remplacer un Use Case applicatif.

Ces responsabilités appartiennent exclusivement aux Domaines et aux APIs.

5. Types de triggers

Les types autorisés sont :

- BEFORE INSERT
- AFTER INSERT
- BEFORE UPDATE
- AFTER UPDATE
- BEFORE DELETE
- AFTER DELETE

Le choix du type doit être justifié dans la documentation.

6. Déclenchement

Un trigger doit être associé à une table précise.

Son exécution doit être prévisible et ne produire aucun effet secondaire inattendu.

Les chaînes de triggers successifs sont fortement déconseillées.

7. Performance

Les triggers doivent :

- exécuter un traitement rapide ;
- éviter les requêtes inutiles ;
- limiter les accès à d'autres tables ;
- préserver les performances des opérations d'écriture.

Les traitements coûteux doivent être déplacés vers les APIs ou les Edge Functions.

8. Sécurité

Les triggers doivent respecter :

- les politiques RLS lorsque cela est applicable ;
- les permissions PostgreSQL ;
- les règles de sécurité définies dans BUILD-SECURITY.

Ils ne doivent jamais contourner les mécanismes de protection des données.

9. Gestion des erreurs

Un trigger doit :

- produire des erreurs explicites ;
- éviter les blocages inutiles ;
- consigner les erreurs techniques lorsque nécessaire.

Les erreurs ne doivent pas divulguer d'informations sensibles.

10. Audit

Les triggers d'audit doivent permettre de tracer :

- l'opération réalisée ;
- la date et l'heure ;
- l'utilisateur concerné lorsque disponible ;
- les objets impactés.

Les journaux d'audit doivent respecter la politique de conservation définie par Vitala.

11. Documentation

Chaque trigger doit être documenté avec :

- son objectif ;
 - la table concernée ;
 - le type de déclenchement ;
 - la fonction appelée ;
 - les éventuels effets secondaires.
-

12. Évolution

Toute création, modification ou suppression d'un trigger doit :

- être réalisée via une migration ;
 - être testée ;
 - être documentée ;
 - préserver la compatibilité lorsque cela est possible.
-

13. Tests

Les triggers doivent être testés afin de vérifier :

- leur bon déclenchement ;
- leur comportement dans les cas limites ;
- leurs performances ;
- l'absence d'effets secondaires inattendus.

Les tests doivent être automatisés lorsque cela est pertinent.

14. Critères de conformité

Un trigger est conforme lorsque :

- il possède une responsabilité technique unique ;
 - il ne contient aucune logique métier complexe ;
 - il respecte les conventions de nommage ;
 - il est documenté ;
 - il est testé ;
 - il ne dégrade pas les performances de la base de données.
-

15. Conclusion

Les triggers PostgreSQL constituent un mécanisme puissant pour automatiser certaines opérations techniques dans Vitala.

Leur utilisation doit rester limitée, transparente et parfaitement documentée afin de préserver une architecture claire, performante et conforme aux principes du Build Blueprint.